National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R2
Recursion & Iteration

Textbook Sections

We have covered so far in the lectures: Sections 1.1.1–1.1.6 and Section 1.2.1 of SICPy. This reflection covers Section 1.2.1. Please read these sections carefully.

Iterative vs Recursive Processes

A function that calls itself gives rise to an *iterative process* if the recursive call is always the last operation that the function carries out. This means that the result of the recursive call is also the result of the function.

Example 1

```
def f(x, y):
    return y if x == 0 else f(x - 1, y + 1)

print(f(4, 5))
```

The body of function `f` has one recursive call. The result of the recursive call is the result of the function. This means that the function gives rise to an iterative process.

Example 2

```
def g(x):
    return 0 if x == 0 else 1 + g(x - 1)

print(g(7))
```

The body of function `g` has one recursive call. The result of the recursive call is not the result of the function. When the recursive call returns, its result needs to be added to 1. This operation is called a "*deferred operation*". The presence of such a deferred operation means that the corresponding process is a *recursive process*.

We say the function gives rise to a recursive process.

Problems:

1. Consider the following implementation of the factorial function:

```
def factorial(n):
    return (1
            if n == 0
            else n * factorial(n - 1))

print(factorial(5))
```

   Is this function giving rise to a recursive or iterative process?

2. In the lecture, it was mentioned that we can "measure" the time taken for a program to run as the number of "operations" required.

   The notion of an "operation" is debatable. Discuss different reasonable definitions of an "operation".

3. How many such operations does it take to compute the result of the function above when applied to the number 5?

4. How much space is required to compute the result of this function when applied to the number 5? Count the number of deferred operations.

5. Consider the following factorial function from the lecture:

```
def factorial(n):
    return fact_iter(1, 0, n)

def fact_iter(product, counter, n):
    return (product
            if counter == n
            else fact_iter(product * (counter + 1), counter + 1, n))

print(factorial(5))
```

How many operations does it take to compute $5!$ with this function?

6. How much space is required to compute the result of the function factorial in Question 5 when applied to the number 5? Count the number of deferred operations. Does this function give rise to a recursive or iterative process?